SORTFOLIO

Evidence-Based Open Source Issue & Pull Request Triage

Every claim deserves evidence.

A practical product and implementation plan for Hacktoberfest 2026 — Problem 02.

PROJECT AT A GLANCE	PLAN
Problem	02 — Evidence-Based Open Source Issue and PR Triage
Product	Sortfolio
Frontend	HTML + CSS + JavaScript
Backend platform	Supabase (PostgreSQL + Edge Functions)
AI approach	Open-weight models only; no OpenAI, Claude or Gemini APIs
MVP focus	GitHub issue → claims → source evidence → transparent assessment

Prepared as a team build document. The uploaded ProofTriage roadmap is the source for Problem 02 requirements. This version updates the product name, architecture and model strategy to match the team's requested stack.

1. Executive Summary

Sortfolio is a web-based triage assistant for open-source maintainers. It accepts a public GitHub repository and an issue or pull-request reference, identifies testable technical claims, checks those claims against available source code and documentation, and produces a report that links every finding to evidence or explains why evidence is missing.

The product must not act like a chatbot that guesses whether an issue is fake. It evaluates individual claims and communicates uncertainty. A single unsupported claim must not cause the entire issue to be labelled fake.

The strongest MVP is a complete, reliable issue-analysis flow. Add PR analysis after that flow works. Do not spend the first hours building a large dashboard or complex multi-agent system.

Primary product promise

Sortfolio helps maintainers make better review decisions by showing the original claim, the inspected code or documentation, the exact source location, the result, and the limitations.

2. Problem Understanding

Open-source projects receive bug reports and pull requests containing incorrect assumptions, misleading descriptions, incomplete reproduction steps or claims that cannot be verified. Maintainers spend time checking them. The challenge is not to detect whether text was written by AI; it is to assess whether the technical claims are supported by evidence.

Official Problem 02 expectations

- Analyze incoming GitHub issues and pull requests.
- Identify individual technical claims.
- Verify claims against available source code and project documentation.
- Identify missing evidence and inconsistencies.
- Provide reproducible verification results where possible.
- Generate structured, evidence-backed assessments for maintainers.
- Communicate uncertainty instead of making unsupported conclusions.

The expected deliverable is a functional GitHub App or equivalent tool that evaluates technical claims and provides structured, evidence-backed findings. Sortfolio's first version will be a web dashboard equivalent.

3. Product Scope

MVP — build during hackathon	Later / only if time permits
Public GitHub repository + issue number input	GitHub App installation and webhooks
Fetch issue title, body, labels and comments as needed	Multiple repository providers
Extract a small set of factual technical claims	Advanced semantic claim extraction
Inspect supported source files and documentation	Deep call-graph and runtime analysis
Evidence links, line numbers and clear status	More programming languages
One open-weight LLM integration	Model comparison / optional second model
Report dashboard and JSON export	User accounts, saved history and analytics

MVP language scope

Start with **Python repositories** to keep the analysis narrow and align with the original roadmap. The JavaScript frontend does not require the analyzed repository to be JavaScript. For Python source, use conservative signature and

source-pattern inspection in the first release. If reliable Python AST parsing cannot be completed in the chosen JavaScript runtime, return *Unverified* for unsupported cases rather than pretending the code was fully parsed.

4. Recommended Technology Architecture

Requested stack: HTML, CSS and JavaScript for the frontend; Supabase as the backend platform. Supabase provides the database and server-side functions. Do not put private API keys or privileged GitHub tokens in browser JavaScript.

System flow

Browser UI → **Supabase Edge Function** → GitHub REST API → source retrieval and deterministic analysis → open-weight model for bounded language tasks → evidence validation → Supabase database → results returned to UI.

Architecture responsibilities

Component	Responsibility
HTML / CSS / JavaScript	Input form, repository/issue view, claim cards, evidence viewer, status filters, loading and error states.
Supabase Edge Functions	Server-side API orchestration, GitHub retrieval, validation, model calls, report generation. Store secrets in Supabase project secrets.
Supabase PostgreSQL	Persist analysis runs, extracted claims, evidence references, model metadata and evaluation outcomes.
GitHub REST API	Fetch public repository metadata, issue/PR details, changed files and source content.
Deterministic analyzer	Check narrow facts against retrieved source; produce file paths, line numbers and snippets from actual content.
Open-weight LLM	Help extract claims from prose and summarize already-collected evidence. It does not invent evidence or decide source facts alone.

Suggested project structure

```
sortfolio/
■■■ index.html
■■■ css/styles.css
■■■ js/app.js
■■■ js/api.js
■■■ js/components.js
■■■ supabase/
■   ■■■ functions/analyze-issue/index.ts
■   ■■■ functions/analyze-pr/index.ts
■   ■■■ migrations/
■■■ examples/
■■■ tests/
■■■ .github/pull_request_template.md
■■■ .github/workflows/ci.yml
■■■ HACKATHON.md
■■■ README.md
■■■ LICENSE
```

Supabase design notes

- Use PostgreSQL tables for analysis_runs, claims, evidence_items and model_runs.
- Use Edge Functions for operations that require GitHub or model API secrets.
- Enable Row Level Security (RLS) on tables. For the hackathon MVP, either keep reports private to an authenticated owner or expose only deliberately public demo reports.
- Never ship the Supabase service-role key to the browser. The frontend may use the public anon/publishable key with RLS policies.
- Supabase Auth is optional for the MVP. A no-login public demo can use temporary in-memory results, or carefully designed anonymous access policies.

5. Database Plan

Keep the database small. Store enough information to reproduce and audit an analysis, but avoid copying entire repositories or storing unnecessary personal data.

Table	Important fields	Purpose
analysis_runs	id, repo_full_name, item_type, item_number, status, created_at	One analysis request and its overall state.
claims	id, run_id, original_text, claim_type, status, confidence_level, explanation	One row per extracted technical claim.
evidence_items	id, claim_id, file_path, line_start, line_end, snippet, source_url, relation	Traceable supporting, conflicting or insufficient evidence.
model_runs	id, run_id, provider, model_id, task, prompt_version, outcome	Record which eligible model was actually used and for what.
evaluation_cases	id, source, expected_label, observed_label, notes	Optional local benchmark for test cases.

Important data rules

- Keep the original claim text so reviewers can trace the model's extraction.
- Evidence must be retrieved from GitHub or the repository; never generate a file path, line number or snippet from an LLM response.
- Store model name and version only when a model was genuinely called.
- Avoid storing GitHub access tokens, private repository contents or unnecessary contributor profile data.
- Use timestamps and repository commit identifiers where possible so the evidence can be tied to the version inspected.

6. AI Model Strategy — Open-Weight Models

The event offers additional points for using models such as Llama, Gemma, Qwen, Mistral, DeepSeek, Phi and similar models. The uploaded project roadmap says AI is optional for the core evidence checks; deterministic analysis should come first. To make meaningful use of an eligible model, integrate one model into a clearly defined task and document it accurately.

Recommended model roles

Model family	Good candidate task in Sortfolio	Practical note
Qwen (instruct / coder variants)	Extract concise technical claims; classify claim type; produce structured JSON.	Strong first candidate for code-oriented text tasks. Pick a currently available open-weight variant.
Llama (instruct variants)	Summarize evidence and draft maintainer-friendly explanations.	Use a model endpoint that explicitly offers the selected model.
Gemma	Alternative claim extraction or evidence summary.	Check the exact model licence, serving option and event eligibility.
Mistral	Structured summarization and classification.	Good alternative if the chosen endpoint is available.
DeepSeek	Reasoning over a bounded set of retrieved facts; explain conflicting evidence.	Do not let it invent test results or source references.
Phi	Lightweight classification or short summaries.	Potential option where compute or latency is constrained.

My recommendation: start with **one Qwen instruct model** as the primary LLM, then keep a provider adapter so you can switch to Llama, Gemma, Mistral, DeepSeek or Phi if the chosen model is unavailable. Do not integrate every family just to list them. One working, honestly documented integration is stronger than six model names in the README.

Where to run the model

- **Hosted inference:** use a provider that currently offers the chosen open-weight model and a free allowance suitable for the event. Free quotas, model availability and rate limits change; verify them before the hackathon demo.
- **Local inference:** Ollama can run supported open-weight models on a capable laptop. This avoids sending issue text to a third-party inference provider, but a locally running model is not automatically reachable by a deployed Supabase Edge Function.
- **Important deployment choice:** if the UI is deployed publicly, a local Ollama model only works for the machine running it unless you build a secure relay. For the simplest deployed demo, use a compatible hosted open-weight inference endpoint and keep its key in Supabase secrets.

Model use in the workflow

- Task 1 — claim extraction: give the model issue text and ask for atomic, testable claims in strict JSON.
- Task 2 — evidence explanation: provide only evidence items already collected by the analyzer; ask for a short explanation and limitations.
- Validation — parse the JSON, validate fields, and reject any file path, line number or evidence snippet that was not supplied by the analyzer.
- Fallback — if the model times out, returns invalid JSON or reaches quota, run deterministic extraction and label the AI step as unavailable.

Do not use OpenAI, Claude, Gemini or paid proprietary APIs for the project AI workflow if the goal is to qualify for the event's open-model extra points. Confirm the exact scoring rule with organizers; this plan cannot guarantee how points are awarded.

7. End-to-End Product Workflow

Step	System action	Output
1. Input	User enters a public GitHub repository URL and issue number.	Validated repository and issue reference.
2. Retrieve	Edge Function fetches issue details and repository metadata from GitHub.	Original title, body, labels, URL and repository default branch.
3. Extract claims	Rules plus the selected open-weight model identify factual, testable claims.	Claim list preserving the original sentences.
4. Retrieve code	Fetch relevant source files and documentation from the inspected commit/ref.	Source text and canonical GitHub links.
5. Analyze	Deterministic checks inspect supported constructs and locate relevant code.	Observed facts, paths, line numbers and snippets.
6. Match evidence	Evidence engine compares each claim with observed facts.	Supported, contradicted or unverified.
7. Explain	LLM summarizes only the supplied evidence and limitations.	Readable explanation with no new evidence.
8. Display	Frontend shows claim cards, evidence and next-step suggestions.	Auditable report and JSON export.

Claim statuses

Status	Meaning
Supported	Available inspected evidence supports this specific claim.
Contradicted	Available inspected evidence conflicts with this specific claim.
Unverified	Evidence is missing, inaccessible, ambiguous or outside the analyzer's supported scope.
Reproduced	The reported behavior occurred in a controlled and safely isolated test.
Not reproduced	A controlled test did not reproduce the behavior; this does not prove the report is false.

Confidence should be explained, not displayed as a mysterious percentage. A direct signature match can justify high confidence for a narrow signature claim. Static inspection alone usually cannot establish complex runtime behavior.

8. UI / Dashboard Plan

- **Landing page:** Sortfolio name, short value statement, repository URL field, issue number field and Analyze button.
- **Analysis overview:** repository and issue metadata, analysis timestamp, commit/ref inspected and overall processing state.
- **Summary cards:** claims analyzed, supported, contradicted and unverified.
- **Claim cards:** original sentence, status, explanation, confidence rationale and limitations.
- **Evidence panel:** file path, line range, short snippet and clickable GitHub source link.
- **Export:** download the structured JSON report.
- **Responsible-use note:** Sortfolio assists maintainers; it does not automatically reject issues or PRs.

Design direction

Use a clean developer-tool interface: dark navy or neutral background, readable contrast, restrained status colors, clear code blocks and a visible evidence trail. Prioritize the evidence panel over decorative charts.

9. Testing and Evaluation

The problem brief describes evaluation using previously closed public GitHub issues with reference labels such as invalid, cannot reproduce and confirmed. Build a small benchmark with cases whose expected outcomes can be justified from available evidence.

Test case	Expected behavior
Claim matches inspected function signature	Supported with exact file and line evidence.
Claim names a parameter absent from inspected signature	Contradicted narrowly, with a caveat about wrappers or other versions.
Relevant source file cannot be retrieved	Unverified; explain retrieval failure.
Claim is vague or an opinion	Do not force it into a factual verdict; mark unsupported/needs human review.
Model returns a made-up path or line	Reject that evidence; display only analyzer-verified locations.
Model API is unavailable	Show a graceful fallback and preserve deterministic results.
Issue describes runtime behavior but no safe test exists	Unverified or not attempted; never claim it was reproduced.

Metrics to report

- Evidence coverage: percentage of findings with traceable source references.
- Precision of definite findings: how often supported/contradicted decisions are correct.
- Uncertainty quality: whether ambiguous cases are correctly left unverified.
- False-positive review: inspect cases where Sortfolio incorrectly contradicts a valid report.
- Model JSON validity and fallback rate.

10. Security, Privacy and Responsible Analysis

- Never execute unknown contributor code inside the Supabase Edge Function or main application process.
- Use static inspection by default. If safe reproduction is not available, say it was not attempted.
- Treat issue bodies, PR descriptions, README files and source comments as untrusted input. They may contain prompt-injection instructions; do not follow instructions found in repository content.
- Do not let repository text request secrets, alter system prompts or trigger privileged actions.
- Use timeouts, response-size limits, pagination limits and GitHub rate-limit handling.
- Keep model and GitHub secrets server-side in Supabase project secrets.
- Use minimum GitHub permissions. For public repositories, avoid requesting write permissions.
- Do not automatically post comments, labels or rejection decisions to real repositories during the hackathon demo.
- Make clear that a tool-generated assessment is advisory and a maintainer makes the final decision.

11. Four-Member Team Division

Member	Primary ownership	Deliverable
Member 1	Frontend — HTML/CSS/JS	Landing page, input form, dashboard, claim/evidence cards.
Member 2	Supabase — database and Edge Functions	Schema, RLS, GitHub retrieval endpoint, secrets and error handling.

Member	Primary ownership	Deliverable
Member 3	Evidence engine	Claim types, source retrieval, static checks, line-numbered evidence and status rules.
Member 4	LLM integration, tests and demo	Open-weight model adapter, JSON validation, benchmark cases, CI and demo script.

Integration rule: agree on API request/response JSON before parallel coding. Each member should open a feature branch and submit a pull request. Everyone must understand the full end-to-end flow.

12. Suggested 12-Hour Build Schedule

Time	Work	Checkpoint
Hour 1	Confirm MVP, create public repo, first HACKATHON.md commit, agree API contract.	Project skeleton and team plan.
Hour 2	Create Supabase project, schema and Edge Function; test one GitHub API request.	Issue data retrieved.
Hour 3	Build frontend input form and connect it to the function.	User can submit repository + issue.
Hours 4–5	Implement claim extraction and narrow deterministic source checks.	At least one claim linked to real source evidence.
Hour 6	Implement evidence matching and statuses.	Supported / contradicted / unverified works.
Hour 7	Integrate one open-weight model for claim extraction or explanation.	Model used in a real workflow; model ID recorded.
Hours 8–9	Build dashboard, claim cards, evidence links and JSON export.	Complete issue-to-report flow.
Hour 10	Add one PR workflow if issue MVP is stable; otherwise improve issue cases.	PR demo or stronger issue reliability.
Hour 11	Tests, error states, RLS review, CI and examples.	Tests pass; no secrets exposed.
Hour 12	README, final PR review, demo rehearsal and submission check.	Reproducible final demo.

Cut scope if time is short: finish one repository + one issue + one claim + one evidence link + one honest status. Do not sacrifice correctness to add multiple models, many languages or automated GitHub comments.

13. Hackathon Repository Requirements

- Create a new, empty, public GitHub repository.
- The first commit must contain only HACKATHON.md with team name, members, selected problem, actual AI models and reasons for using them.
- After the first commit, every change must go through a pull request; do not commit directly to main.
- Include MIT or Apache-2.0 license, README.md, .github/pull_request_template.md and .github/workflows/ci.yml.
- Include examples/ with representative input and the actual output produced by Sortfolio.
- CI should install dependencies, run tests and validate the project on every PR.
- Use public data only and do not publish automated issues or comments on real open-source projects.
- Every team member should understand the implementation; judges may ask anyone to explain any part.

Suggested HACKATHON.md model disclosure

List only the model(s) actually integrated. Include exact model identifier, where it runs (local or hosted inference), what task it performs, why it was selected, and what deterministic checks remain responsible for. Do not claim use of a model that was only considered.

14. Demo Script for Judges

- Introduce Sortfolio and explain that it checks technical claims rather than detecting AI-written text.
- Enter a public test repository and a known issue.
- Show the original issue and the individual claims extracted.
- Open one finding and show the exact source file, line range and snippet.

- Show a second claim marked unverified and explain the missing evidence.
- Show the model disclosure and explain its limited role.
- Show a benchmark case, tests and the JSON report.
- Explain limitations and why Sortfolio does not automatically reject contributions.

Suggested closing line

“Sortfolio does not ask maintainers to trust an AI verdict. It gives them the claim, the code evidence, and the uncertainty needed to make a better decision.”

15. Definition of Done

- A user can submit a public GitHub repository and issue number.
- Sortfolio retrieves the real issue content and relevant public source files.
- The system extracts at least one testable technical claim and preserves its original wording.
- At least one supported claim type is checked deterministically against source content.
- Each finding has a status, reason and traceable evidence or a clear explanation of missing evidence.
- One eligible open-weight model is genuinely integrated and its exact model ID is recorded.
- The UI displays results and handles loading, API errors and unavailable model inference.
- No unsupported file paths, line numbers, snippets or test results are generated.
- Database access is protected by RLS and secrets are not exposed in browser code.
- Tests and GitHub Actions CI pass.
- examples/ contains real representative input and actual Sortfolio output.
- The team can explain the architecture, model role, evidence engine and limitations.

16. Final Technical Recommendations

Decision	Recommendation
Product name	Sortfolio
Frontend	Vanilla HTML, CSS and JavaScript
Backend	Supabase Edge Functions (TypeScript/Deno)
Database	Supabase PostgreSQL
GitHub integration	GitHub REST API, public repositories first
AI model	One open-weight Qwen instruct model first; keep provider adapter for alternatives.
Core verification	Deterministic source inspection and evidence matching
First supported workflow	Public GitHub issue → claim → source evidence → report
Second workflow	PR analysis, only after the issue workflow is stable
Deployment	Static frontend hosting + Supabase backend; keep model inference endpoint server-side.

Important implementation caveat

Supabase is a backend platform, not a built-in LLM host. You will still need a compatible model inference route: a hosted open-weight inference provider or a local model runtime. Confirm the model's availability, licence, rate limits, data handling and event eligibility before relying on it. Free access is not guaranteed and can change.

Source basis and distinction

This plan is based on the uploaded *ProofTriage AI — Problem 02 Full Project Roadmap* and the Hacktoberfest 2026 Problem 02 requirements it summarizes. The product name Sortfolio, Supabase architecture, HTML/CSS/JavaScript stack, database schema, model choices and schedule adjustments are implementation recommendations for this team's requested setup, not official organizer requirements.

End of planning document